MANUAL TÉCNICO
DEL DESARROLLADOR

Sistema de Gestión de Usuarios
Aplicación Web Multicapa J2EE

MF0227_3 · Programación orientada a objetos
UF2405 · Modelo de programación web y bases de datos

UD1 · Comparativa .NET y J2EE / Despliegue de Arquitectura Multicapa

VERSIÓN 1.0 | OCTUBRE 2026
Documento técnico interno

Ficha del documento

Módulo formativo: MF0227_3 - Programación orientada a objetos

Unidad formativa: UF2405 - Modelo de programación web y bases de datos

Actividad: UD1 - Comparativa .NET y J2EE / Despliegue de Arquitectura Multicapa

Autor: [Tu Nombre y Apellidos]

Fecha: Octubre 2026

Versión: 1.0

Clasificación: Documento técnico interno

Índice

1. Introducción y objetivos

2. Arquitectura del sistema

3. Stack tecnológico

4. Estructura del proyecto

5. Descripción de capas

6. Código fuente comentado

7. Flujo de una petición HTTP

8. Configuración y despliegue

9. Base de datos H2

10. Patrones de diseño aplicados

11. Troubleshooting técnico

12. Extensibilidad y mantenimiento

13. Referencias bibliográficas

Apéndice A. Glosario técnico

Apéndice B. Checklist de despliegue

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Propósito del Documento

Este manual técnico está dirigido a desarrolladores que necesiten comprender, mantener o extender la aplicación web multicapa desarrollada como práctica de la UF2405. Documenta la arquitectura, decisiones de diseño, código fuente y mecanismos de despliegue del sistema.

1.2. Objetivos del Desarrollo

Implementar una arquitectura web multicapa siguiendo el patrón MVC (Modelo-Vista-Controlador)

Utilizar tecnologías J2EE (Java EE) como estándar de facto

Aplicar el principio de Separación de Responsabilidades (SoC)

Garantizar portabilidad y aislamiento del entorno de ejecución

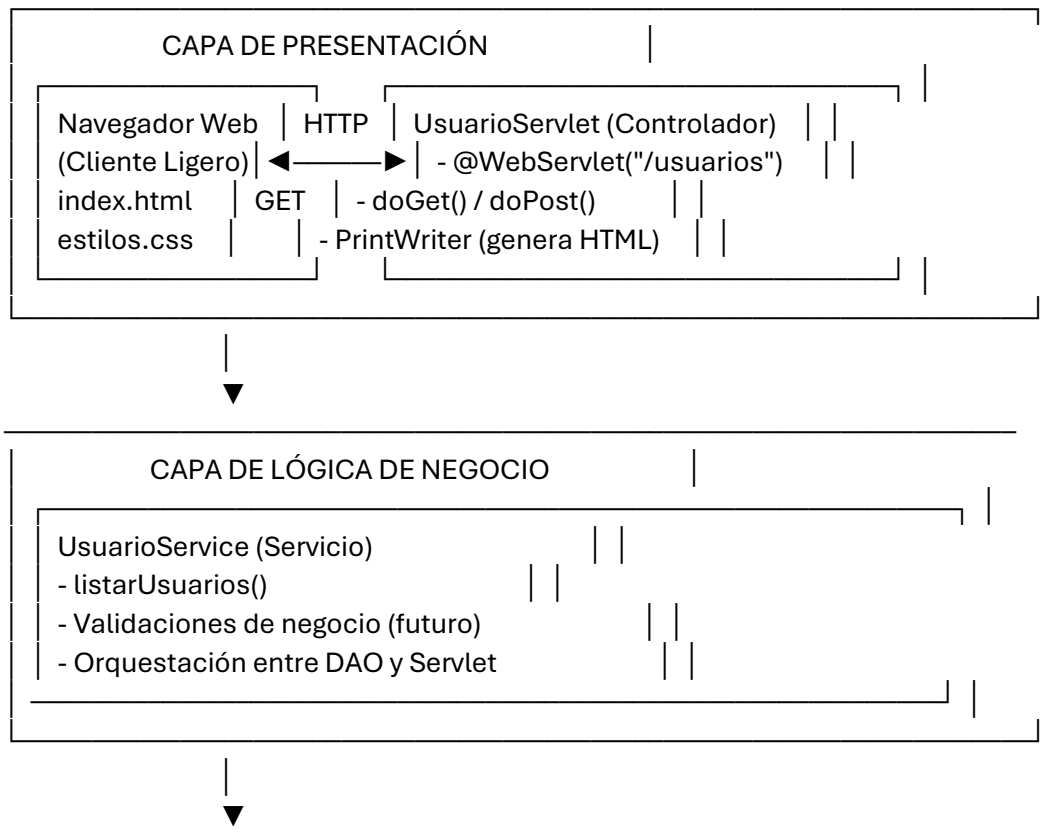
Cumplir los criterios de evaluación CE1.1, CE1.2, CE1.3 y CE1.7 de la UF2405

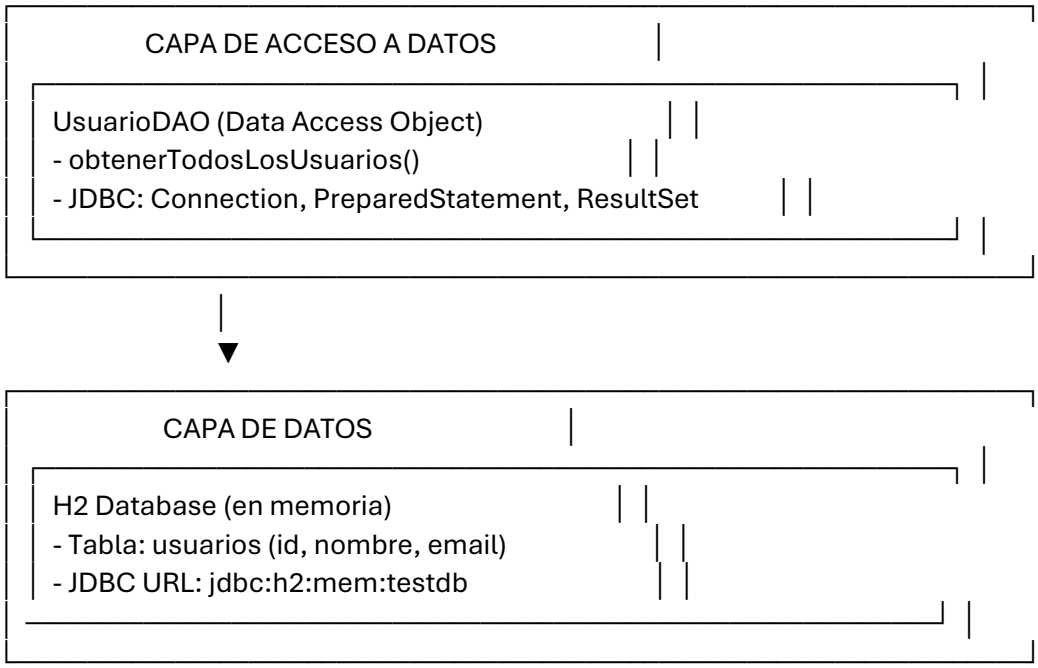
1.3. Alcance

El sistema implementa un CRUD básico de gestión de usuarios, demostrando el flujo completo desde la capa de presentación (navegador) hasta la capa de datos (H2 en memoria), pasando por las capas de negocio y acceso a datos.

2. ARQUITECTURA DEL SISTEMA

2.1. Diagrama de Arquitectura Multicapa





2.2. Principios Arquitectónicos Aplicados

Principio	Implementación
Separación de Responsabilidades	Cada capa tiene una única responsabilidad
Inversión de Dependencias	Las capas superiores dependen de abstracciones
Patrón MVC	Servlet (Controlador), HTML (Vista), Usuario.java (Modelo)
Patrón DAO	Aislamiento del acceso a datos en UsuarioDAO
Patrón Service	Lógica de negocio encapsulada en UsuarioService

3. STACK TECNOLÓGICO

3.1. Tecnologías Principales

Tecnología	Versión	Función	Justificación
Java JDK	25.0.4.1 (Temurin)	Lenguaje de programación	Estándar J2EE, multiplataforma
Apache Maven	3.9.6	Gestión de dependencias y build	Estándar de facto en Java

Apache Tomcat	7.0.47 (embebido)	Servidor de aplicaciones	Contenedor Servlet ligero
H2 Database	2.2.224	Base de datos relacional	Embebida, sin instalación
Servlet API	3.1.0	API de servlets Java EE	Estándar J2EE
HTML5	-	Marcado de la vista	Estándar web
CSS3	-	Estilos de presentación	Separación presentación/lógica

3.2. Dependencias Maven (`pom.xml`)

```
<dependencies>
  <!-- API de Servlets (provided: la provee Tomcat) -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
    <scope>provided</scope>
  </dependency>

  <!-- Base de datos H2 embebida -->
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <version>2.2.224</version>
  </dependency>
</dependencies>
```

4. ESTRUCTURA DEL PROYECTO

4.1. Árbol de Directorios (Estándar Maven)

```
proyecto-web-multicapa/
│
├── pom.xml                    # Descriptor Maven
├── instalar_maven_local.bat       # Script instalación local
├── lanzar_proyecto.bat            # Script de ejecución
├── abrir_terminal.bat             # Script terminal
│
├── src/
│   └── main/
│       ├── java/
│       │   └── com/empresa/proyecto/
│       │       ├── model/
│       │       │   └── Usuario.java      # JavaBean (Modelo)
```

```
│   │   ├── dao/
│   │   │   └── UsuarioDAO.java     # Acceso a datos
│   │   ├── service/
│   │   │   └── UsuarioService.java  # Lógica de negocio
│   │   └── web/
│   │       └── UsuarioServlet.java  # Controlador
│   │
│   └── webapp/
│       ├── WEB-INF/
│       │   └── web.xml              # Descriptor despliegue
│       ├── css/
│       │   └── estilos.css          # Hoja de estilos
│       └── index.html               # Página principal
│
├── tools/
│   └── apache-maven-3.9.6/          # Maven local (no afecta SO)
│
└── target/                      # Generado por Maven
    ├── classes/
    └── tomcat/
```

4.2. Convenciones de Nomenclatura

Paquetes Java: com.empresa.proyecto.[capa]

Clases: PascalCase (UsuarioDAO, UsuarioService)

Métodos: camelCase (obtenerTodosLosUsuarios())

Variables: camelCase (listaUsuarios)

Constantes: UPPER_SNAKE_CASE (no utilizadas en este proyecto)

Archivos web: kebab-case (estilos.css)

5. DESCRIPCIÓN DE CAPAS

5.1. Capa de Presentación (Vista + Controlador)

Responsabilidad: Interactuar con el usuario, recibir peticiones HTTP y generar respuestas HTML.

Componentes:

index.html: Página estática de entrada

estilos.css: Definición visual

UsuarioServlet.java: Controlador que procesa peticiones

Tecnología: Servlet API 3.1, HTML5, CSS3

5.2. Capa de Lógica de Negocio (Servicio)

Responsabilidad: Encapsular reglas de negocio, validar datos y orquestar llamadas al DAO.

Componentes:

UsuarioService.java: Servicio que expone operaciones de negocio

Principio: La capa de presentación NUNCA debe llamar directamente al DAO.

5.3. Capa de Acceso a Datos (DAO)

Responsabilidad: Abstraer el acceso a la base de datos, implementar JDBC.

Componentes:

UsuarioDAO.java: Implementa operaciones CRUD

Patrón: DAO (Data Access Object) - desacopla la lógica de negocio del SGBD.

5.4. Capa de Modelo (Entidad)

Responsabilidad: Representar las entidades del dominio.

Componentes:

Usuario.java: JavaBean con propiedades privadas y getters

Patrón: JavaBean (serializable, constructor vacío, getters/setters).

6. CÓDIGO FUENTE COMENTADO

6.1. `model/Usuario.java` - JavaBean

```
package com.empresa.proyecto.model;

import java.io.Serializable;

/**
 * JavaBean que representa la entidad Usuario.
 * Implementa Serializable para permitir persistencia en sesión HTTP.
 *
 * @author [Tu Nombre]
 * @version 1.0
 */
public class Usuario implements Serializable {

   private static final long serialVersionUID = 1L;

   private int id;
   private String nombre;
   private String email;

   /**
    * Constructor completo.
    */
```

```
    public Usuario(int id, String nombre, String email) {
        this.id = id;
        this.nombre = nombre;
        this.email = email;
    }

    // Getters (no se incluyen setters porque es solo lectura en este ejemplo)
    public int getId() { return id; }
    public String getNombre() { return nombre; }
    public String getEmail() { return email; }
}
```

6.2. `dao/UsuarioDAO.java` - Acceso a Datos

```
package com.empresa.proyecto.dao;

import com.empresa.proyecto.model.Usuario;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * Data Access Object para la entidad Usuario.
 * Encapsula toda la lógica de acceso a la base de datos H2.
 *
 * En producción, reemplazar la simulación por código JDBC real
 * (comentado en el método).
 */
public class UsuarioDAO {

    /**
     * Obtiene todos los usuarios de la base de datos.
     * @return Lista de objetos Usuario
     */
    public List<Usuario> obtenerTodosLosUsuarios() {
        List<Usuario> lista = new ArrayList<>();

        /*
        // === CÓDIGO JDBC REAL (para producción con H2/MySQL) ===
        String url = "jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1";
        String user = "sa";
        String pass = "";

        try {
            Class.forName("org.h2.Driver");
            Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement ps = conn.prepareStatement(
                "SELECT id, nombre, email FROM usuarios"
            );
```

```
        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
          lista.add(new Usuario(
            rs.getInt("id"),
            rs.getString("nombre"),
            rs.getString("email")
          ));
        }

        rs.close();
        ps.close();
        conn.close();
      } catch (SQLException e) {
        e.printStackTrace();
      }
      */

      // Simulación para demostración (sin dependencias externas)
      lista.add(new Usuario(1, "Ana García", "ana@empresa.com"));
      lista.add(new Usuario(2, "Carlos López", "carlos@empresa.com"));
      lista.add(new Usuario(3, "María Ruiz", "maria@empresa.com"));

      return lista;
    }
}
```

6.3. `service/UsuarioService.java` - Lógica de Negocio

```
package com.empresa.proyecto.service;

import com.empresa.proyecto.dao.UsuarioDAO;
import com.empresa.proyecto.model.Usuario;
import java.util.List;

/**
 * Capa de servicio que encapsula la lógica de negocio.
 * Actúa como intermediario entre el Servlet (presentación) y el DAO (datos).
 */
public class UsuarioService {

    private UsuarioDAO usuarioDAO;

    public UsuarioService() {
        this.usuarioDAO = new UsuarioDAO();
    }

    /**
     * Lista todos los usuarios aplicando reglas de negocio.
```

```
     * Aquí se podrían añadir validaciones, filtros, etc.
     */
    public List<Usuario> listarUsuarios() {
        // Ejemplo de regla de negocio: solo devolver usuarios activos
        // (en este caso, todos lo son)
        return usuarioDAO.obtenerTodosLosUsuarios();
    }
}
```

6.4. `web/UsuarioServlet.java` - Controlador

```
package com.empresa.proyecto.web;

import com.empresa.proyecto.model.Usuario;
import com.empresa.proyecto.service.UsuarioService;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;

/**
 * Servlet que actúa como Controlador en el patrón MVC.
 * Mapeado a la URL "/usuarios" mediante anotación.
 */
@WebServlet("/usuarios")
public class UsuarioServlet extends HttpServlet {

    private UsuarioService usuarioService;

    /**
     * Inicialización del servlet (se ejecuta una vez al cargar).
     */
    @Override
    public void init() throws ServletException {
        usuarioService = new UsuarioService();
    }

    /**
     * Maneja peticiones HTTP GET.
     */
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
```

```
        // 1. Obtener datos del modelo (vía servicio)
        List<Usuario> usuarios = usuarioService.listarUsuarios();

        // 2. Configurar respuesta HTTP
        response.setContentType("text/html;charset=UTF-8");

        // 3. Generar vista dinámica (HTML)
        PrintWriter out = response.getWriter();
        out.println("<!DOCTYPE html>");
        out.println("<html><head><title>Lista de Usuarios</title>");
        out.println("<link rel='stylesheet' href='css/estilos.css'>");
        out.println("</head><body>");
        out.println("<div class='container'>");
        out.println("<h2>Usuarios registrados</h2>");
        out.println("<ul class='lista-usuarios'>");

        for (Usuario u : usuarios) {
            out.println("<li><strong>" + u.getNombre() +
                    "</strong> - " + u.getEmail() + "</li>");
        }

        out.println("</ul>");
        out.println("<a href='index.html' class='btn'>Volver</a>");
        out.println("</div></body></html>");
    }
}
```

7. FLUJO DE UNA PETICIÓN HTTP

7.1. Secuencia de una Petición GET /usuarios

1. Navegador → Servidor Tomcat
   GET /proyecto/usuarios HTTP/1.1
   Host: localhost:8080

2. Tomcat → UsuarioServlet
   - Identifica el servlet mapeado a "/usuarios"
   - Crea objetos HttpServletRequest y HttpServletResponse
   - Llama a service() → doGet()

3. UsuarioServlet.doGet()
   - Instancia UsuarioService (si no existe)
   - Llama a usuarioService.listarUsuarios()

4. UsuarioService.listarUsuarios()
   - Instancia UsuarioDAO (si no existe)
   - Llama a usuarioDAO.obtenerTodosLosUsuarios()

5. UsuarioDAO.obtenerTodosLosUsuarios()

- Establece conexión JDBC con H2
- Ejecuta SELECT * FROM usuarios
- Mapea ResultSet a List<Usuario>
- Cierra conexión

6. Retorno en cascada
DAO → Service → Servlet

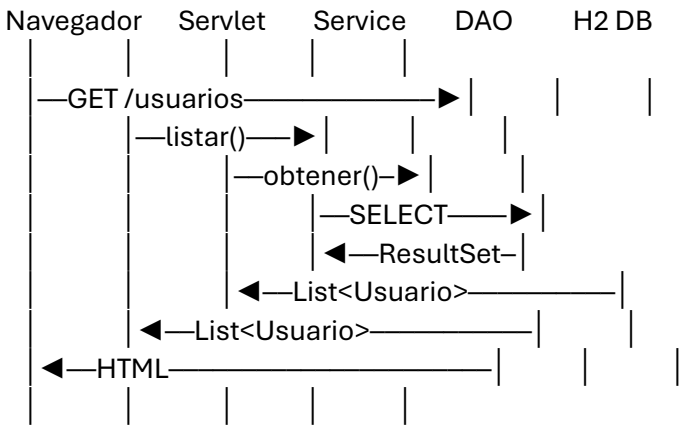
7. UsuarioServlet.doGet()
- Genera HTML dinámico con PrintWriter
- Establece Content-Type: text/html

8. Servidor Tomcat → Navegador
HTTP/1.1 200 OK
Content-Type: text/html;charset=UTF-8

<html>...</html>

9. Navegador
- Renderiza HTML
- Aplica estilos CSS
- Muestra lista de usuarios

7.2. Diagrama de Secuencia UML (Texto)



8. CONFIGURACIÓN Y DESPLIEGUE

8.1. Requisitos del Entorno de Desarrollo

Componente	Versión Mínima	Notas
JDK	8+	Recomendado 11 o 17
Maven	3.6+	Se instala localmente en tools/
IDE	Cualquiera	IntelliJ, Eclipse, VS Code
Navegador	Cualquiera	Chrome, Firefox, Edge

8.2. Compilación Manual

```
# Navegar al directorio del proyecto
cd proyecto-web-multicapa

# Configurar Maven local (si no está en PATH)
export PATH=$PATH:$(pwd)/tools/apache-maven-3.9.6/bin  # Linux/Mac
set PATH=%PATH%;%cd%\tools\apache-maven-3.9.6\bin      # Windows

# Compilar
mvn clean compile

# Empaquetar WAR
mvn package

# Ejecutar en Tomcat embebido
mvn tomcat7:run
```

8.3. Variables de Entorno Relevantes

Variable	Valor	Propósito
JAVA_HOME	Ruta al JDK	Requerido por Maven
PATH	Incluye %JAVA_HOME%\bin	Acceso a java y javac
MAVEN_OPTS	-Xmx512m (opcional)	Memoria para Maven

8.4. Puertos Utilizados

Puerto	Servicio	Protocolo
8080	Tomcat HTTP	HTTP/1.1
8009	AJP (opcional)	AJP/1.3

9. BASE DE DATOS H2

9.1. Características

Tipo: Relacional embebida

Modo: En memoria (mem:testdb)

Persistencia: Solo durante la sesión (se pierde al cerrar)

Driver: org.h2.Driver

URL JDBC: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1

9.2. Esquema de Base de Datos

```
CREATE TABLE IF NOT EXISTS usuarios (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(50) NOT NULL,
  email VARCHAR(50) NOT NULL UNIQUE
);

INSERT INTO usuarios (nombre, email) VALUES
('Ana García', 'ana@empresa.com'),
('Carlos López', 'carlos@empresa.com'),
('María Ruiz', 'maria@empresa.com');
```

9.3. Migración a MySQL/PostgreSQL

Para migrar a un SGBD externo, modificar UsuarioDAO.java:

```
// Cambiar URL y credenciales
String url = "jdbc:mysql://localhost:3306/gestion?useSSL=false";
String user = "root";
String pass = "password";

// Cambiar driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Añadir dependencia en pom.xml
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
```

10. PATRONES DE DISEÑO APLICADOS

10.1. Modelo-Vista-Controlador (MVC)

Componente MVC	Implementación	Responsabilidad
Modelo	Usuario.java	Representa datos
Vista	HTML generado por PrintWriter	Presentación
Controlador	UsuarioServlet.java	Coordina flujo

10.2. Data Access Object (DAO)

Propósito: Desacoplar la lógica de negocio del acceso a datos.

Beneficios:

Cambio de SGBD sin modificar lógica de negocio

Facilita testing (mock del DAO)

Centraliza consultas SQL

10.3. Service Layer

Propósito: Encapsular lógica de negocio compleja.

Beneficios:

Reutilización de lógica entre distintos controladores

Transacciones distribuidas (futuro)

Validaciones centralizadas

10.4. Front Controller (Variante)

El UsuarioServlet actúa como Front Controller para la ruta /usuarios, centralizando el manejo de peticiones.

11. TROUBLESHOOTING TÉCNICO

11.1. Errores Comunes y Soluciones

Error	Causa	Solución	
ClassNotFoundException: org.h2.Driver	Driver H2 no en classpath	Verificar dependencia en pom.xml	
HTTP 404 - /proyecto/	No existe index.html en webapp/	Crear archivo o acceder a /proyecto/usuarios	
Port 8080 already in use	Otro proceso usando el puerto	`netstat -ano \	findstr :8080` y matar proceso
mvn: command not found	Maven no en PATH	Usar script instalar_maven_local.bat	
UnsupportedClassVersionError	JDK incompatible	Usar JDK 11+ o cambiar maven.compiler.source	
Servlet.init() exception	Error en init() del servlet	Revisar logs de Tomcat en target/tomcat/logs	

11.2. Logs y Depuración

Activar logs de Tomcat:

```
<!-- En pom.xml, dentro del plugin tomcat7-maven-plugin -->
<configuration>
  <port>8080</port>
  <path>/proyecto</path>
  <systemProperties>
```

```
    <org.apache.catalina.level>FINE</org.apache.catalina.level>
  </systemProperties>
</configuration>
```

Depuración con IDE:

```
mvnDebug tomcat7:run
# Luego conectar debugger en puerto 8000
```

11.3. Rendimiento

Pool de conexiones: En producción, usar HikariCP o Tomcat JDBC Pool

Cache: Implementar cache en UsuarioService para consultas frecuentes

CDN: Servir CSS/JS desde CDN en producción

12. EXTENSIBILIDAD Y MANTENIMIENTO

12.1. Añadir un Nuevo Endpoint (Ejemplo: crear usuario)

Paso 1: Añadir método en UsuarioDAO:

```
public void crearUsuario(Usuario u) {
  // INSERT INTO usuarios ...
}
```

Paso 2: Añadir método en UsuarioService:

```
public void registrarUsuario(Usuario u) {
  // Validaciones
  if (u.getEmail() == null || u.getEmail().isEmpty()) {
    throw new IllegalArgumentException("Email requerido");
  }
  usuarioDAO.crearUsuario(u);
}
```

Paso 3: Añadir método en UsuarioServlet:

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
  String nombre = req.getParameter("nombre");
  String email = req.getParameter("email");
  usuarioService.registrarUsuario(new Usuario(0, nombre, email));
  resp.sendRedirect("usuarios");
}
```

12.2. Migrar a JSP/Thymeleaf

Reemplazar PrintWriter por motor de plantillas:

```
// En lugar de PrintWriter
RequestDispatcher dispatcher = request.getRequestDispatcher("/usuarios.jsp");
request.setAttribute("usuarios", usuarios);
dispatcher.forward(request, response);
```

12.3. Añadir Seguridad (Futuro)

Autenticación: Java EE Security API o Spring Security

Autorización: Roles y permisos por URL

HTTPS: Configurar SSL en Tomcat

CSRF: Tokens en formularios

12.4. Testing

Unit Testing (JUnit 5 + Mockito):

```
@Test
void testListarUsuarios() {
  UsuarioDAO mockDAO = mock(UsuarioDAO.class);
  when(mockDAO.obtenerTodosLosUsuarios())
    .thenReturn(Arrays.asList(new Usuario(1, "Test", "test@test.com")));

  UsuarioService service = new UsuarioService(mockDAO);
  List<Usuario> result = service.listarUsuarios();

  assertEquals(1, result.size());
}
```

Integration Testing:

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment = RANDOM_PORT)
public class UsuarioServletIT {
  @Test
  public void testGetUsuarios() {
    // Test HTTP real contra Tomcat embebido
  }
}
```

13. REFERENCIAS BIBLIOGRÁFICAS

13.1. Documentos de la UF2405

UD1: Introducción al desarrollo web - Arquitectura cliente/servidor, protocolos HTTP/TCP-IP

UD2: Arquitectura multicapa - Modelo de 3 capas, patrón MVC

UD3: Capa de presentación - Servlets, JSP, JavaBeans

UD5: Acceso a bases de datos - JDBC, drivers, conexiones, transacciones

13.2. Especificaciones y Estándares

Java Servlet Specification 3.1 - Oracle

JDBC 4.2 API - Oracle

HTML5 Specification - W3C

Maven POM Reference - Apache Software Foundation

13.3. Bibliografía Recomendada

Core Servlets and JavaServer Pages - Marty Hall & Larry Brown

Java for the Web with Servlets, JSP, and EJB - Budi Kurniawan

Effective Java - Joshua Bloch

Clean Architecture - Robert C. Martin

APÉNDICE A: GLOSARIO TÉCNICO

Término	Definición
Servlet	Clase Java que extiende capacidades del servidor web
DAO	Patrón que abstrae el acceso a datos
JavaBean	Clase Java serializable con constructor vacío y getters/setters
JDBC	API estándar de Java para acceso a bases de datos
MVC	Patrón Modelo-Vista-Controlador
WAR	Web Application Archive - formato de empaquetado Java EE
Context Path	Ruta base de la aplicación (/proyecto)
Driver	Implementación específica de JDBC para un SGBD

APÉNDICE B: CHECKLIST DE DESPLIEGUE

☐ JDK instalado y `JAVA_HOME` configurado

☐ Maven local en carpeta `tools/`

☐ Estructura de carpetas según estándar Maven

☐ `pom.xml` con dependencias correctas

☐ Código Java compilado sin errores (`mvn compile`)

☐ `index.html` presente en `src/main/webapp/`

☐ Puerto 8080 libre

☐ Navegador accede a `http://localhost:8080/proyecto`

Fin del Manual Técnico del Desarrollador

Documento generado para la Unidad Formativa UF24